

Lecture 12: Binary Search & Monotonic Search Spaces

1. Core Paradigm: The Monotonic Search Space

Binary Search is a highly optimized searching algorithm, but it comes with a strict mathematical prerequisite.

- **The Monotonic Condition:** Binary search is strictly applicable only on monotonic search spaces. A sequence is monotonic if it is entirely non-increasing (sorted in descending order) or entirely non-decreasing (sorted in ascending order). If an array's values fluctuate up and down, binary search will fundamentally fail.
- **Search Space Halving:** Unlike Linear Search, which evaluates every element one by one from index 0 to $N-1$, Binary Search checks the exact middle of the array. Based on that single comparison, it completely discards half of the remaining array, halving the search space on every single iteration.

2. The Integer Overflow Trap (Crucial Edge Case)

In a standard 32-bit integer environment, the absolute maximum value a signed integer can hold is $2^{31} - 1$ (`INT_MAX`).

- **The Naive Formula:** The standard mathematical midpoint is calculated as `mid = (start + end) / 2`.
- **The Overflow Threat:** If you have a massive array where both `start` and `end` are close to $2^{31} - 1$, adding them together (`start + end`) will instantly exceed the 32-bit limit, causing an integer overflow error before the division by 2 can even occur.
- **The Optimized Algebraic Formula:** To prevent this, we calculate the mathematical difference first, divide it, and then add it to the base index. The strictly safe formula used in competitive programming is:

$$mid = start + \frac{(end - start)}{2}$$

3. Mathematical Complexity Derivation (Geometric Progression)

We can prove the time complexity of Binary Search by tracking the size of the search space N through a Geometric Progression.

1. Iteration 1: Size is N
2. Iteration 2: Size is $N/2$
3. Iteration 3: Size is $N/4$
4. Iteration k : Size is $N/2^k$

The worst-case scenario occurs when the search space is reduced to exactly 1 element. Therefore, we equate:

$$\frac{N}{2^k} = 1$$
$$N = 2^k$$

Applying $\log_2$ to both sides mathematically yields the Time Complexity:

$$\log_2(N) = k \implies O(\log N)$$

Search Algorithms Comparison Table

Metric	Linear Search	Binary Search
Prerequisite	None (Works on unsorted arrays)	Strictly Monotonic (Sorted arrays)
Time Complexity	$O(N)$ (e.g., 1000 comparisons for 1000 items)	$O(\log N)$ (e.g., ≈ 10 comparisons for 1000 items)
Space Complexity	$O(1)$	$O(1)$

Problem-Solving Deconstruction

Standard Binary Search Implementation

- 1. Problem Statement & Constraints:** Given a strictly sorted array of integers and a key value, return the index of the key if it exists. If the key does not exist in the array, return -1.
- 2. Core Intuition:** Establish two pointers bounding the search space (start at 0, end at $N-1$). Calculate the mid index. If the element at mid is equal to the key, the search is complete. If the key is strictly greater than $\text{arr}[\text{mid}]$, the key must mathematically exist in the right half of the array, so we advance $\text{start} = \text{mid} + 1$. If the key is smaller, we pull back $\text{end} = \text{mid} - 1$.
- 3. Algorithmic Steps:**
 1. Initialize bounding pointers $\text{start} = 0$ and $\text{end} = \text{size} - 1$.
 2. Calculate the safe midpoint: $\text{mid} = \text{start} + (\text{end} - \text{start}) / 2$.
 3. Open a while loop bound by $\text{start} \leq \text{end}$.
 4. **Match Evaluated:** if $(\text{arr}[\text{mid}] == \text{key})$, return mid.
 5. **Right Traversal:** if $(\text{key} > \text{arr}[\text{mid}])$, update $\text{start} = \text{mid} + 1$.
 6. **Left Traversal:** else, update $\text{end} = \text{mid} - 1$.
 7. Recalculate $\text{mid} = \text{start} + (\text{end} - \text{start}) / 2$ at the bottom of the loop.
 8. If the loop breaks (meaning start crossed end), return -1 (Not Found).

4. Dry Run:

Input: `arr = [3, 8, 11, 14, 16]`, `key = 14` (Size $N = 5$)

1. `start = 0`, `end = 4`. `mid = 0 + (4-0)/2 = 2`.
2. Check `arr[2] → 11`. Is `11 == 14`? No.
3. Is `14 > 11`? Yes. Discard left half. Update `start = mid + 1 = 3`.
4. `start = 3`, `end = 4`. Recalculate `mid = 3 + (4-3)/2 = 3`.
5. Check `arr[3] → 14`. Is `14 == 14`? Yes.
6. Return index `3`.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\log N)$	The search space explicitly halves on every iteration.
Space Complexity	$O(1)$	Operates entirely in-place utilizing three scalar tracker pointers (<code>start</code> , <code>end</code> , <code>mid</code>).

C++

```
#include<iostream>
using namespace std;

int binarySearch(int arr[], int size, int key) {
    int start = 0;
    int end = size - 1;

    // Optimized midpoint calculation to prevent INT_MAX overflow
    int mid = start + (end - start) / 2;

    while (start <= end) {

        // Exact match found
        if (arr[mid] == key) {
            return mid;
        }

        // Navigate to the Right sub-array
        if (key > arr[mid]) {
            start = mid + 1;
        }
        // Navigate to the Left sub-array
        else {
            end = mid - 1;
        }

        // Recalculate midpoint for the new boundaries
        mid = start + (end - start) / 2;
    }
}
```

```
    }

    // Key not found in array
    return -1;
}

int main() {
    int even[6] = {2, 4, 6, 8, 12, 18};
    int odd[5] = {3, 8, 11, 14, 16};

    int evenIndex = binarySearch(even, 6, 6);
    cout << "Index of 6 is " << evenIndex << endl;

    int oddIndex = binarySearch(odd, 5, 14);
    cout << "Index of 14 is " << oddIndex << endl;

    return 0;
}
```